

Summative Assessment Task TOR

Ain Shams University
Faculty of Engineering



University of
East London

Course Code: MCT 313

Automation – Major Task

Phase 1

Task Description:

In groups of 3 students each, you are required to:

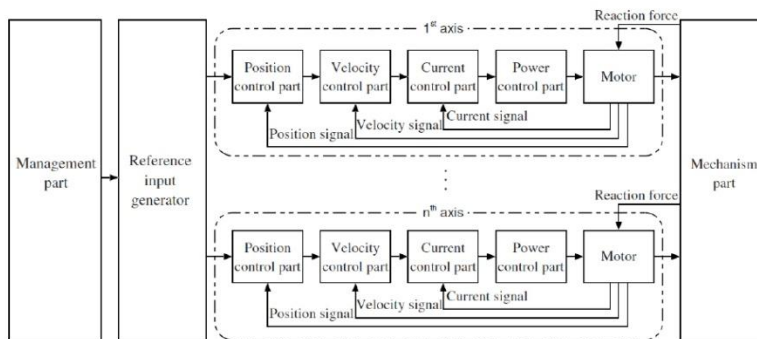
- Use SIMATIC S7-1200 PLC or PLCSIM to control one of the following industrial systems:
 - An Industrial liquid-level system
 - An industrial mechatronic single axis servo system using a cascaded PID control system.
- Design and tune the controllers using the PID_Compact block in TIA Portal
- You may use Factory IO/MATLAB Simulink for simulating the plants and SIL/HIL simulation.
- A suitable HMI interface on TIA portal to monitor and visualize the systems' states readings as well as the desired set-points and system faults
- Utilize field-related alarms for the detection of hardware and sequence faults.

Deliverables:

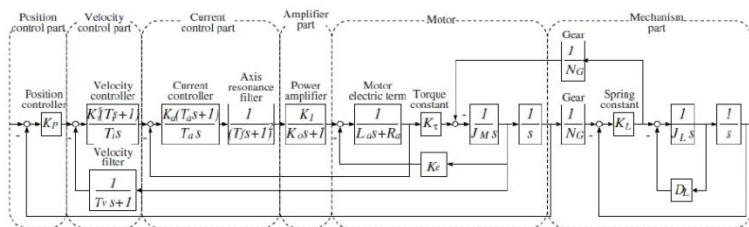
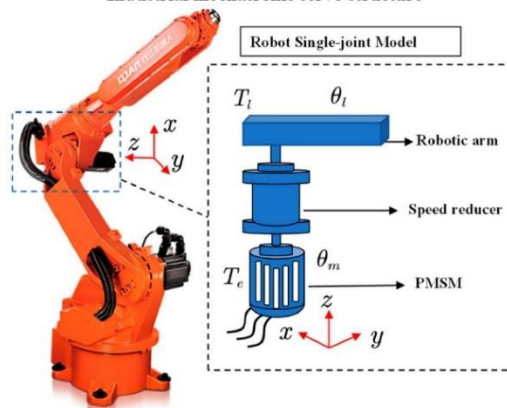
- Software TIA portal code
- HMI interface on TIA portal
- SIL/HIL simulation of the entire control system while the PLC/PLCSIM is running the plants' controllers
- A technical report of the task highlighting your results and discussing your developed system performance. The report shall include the contribution of each team member in performing all task activities in a tabular form.



Liquid Level System in Factory I/O



Industrial mechatronic servo structure



Construction of servo control system of a single-axis mechatronic motion unit

Submission: Week 11

Task Description:

In groups of 3 students each, you are required to:

- Design a full functioning production line on a simulation platform (ex. FACTORY I/O)
- Use Siemens TIA portal software in order to control the simulated production line while incorporating the studied functions in the design process of such application.
- Design a suitable HMI module in order to monitor and visualize the various states of the simulated production line and supply supervisory control commands
- Utilize field-related alarms for the detection of hardware and sequence faults.

The mentioned production line performs the tasks of machining, sorting and assembly starting from raw materials to finished assembled products.

Production Line Components:

1. Feeding unit:

This unit can be considered as the beginning of your production line, where it generates the random raw materials. It also incorporates a pick & place robot which can be used to place the fed part to the beginning of your line.



Fig. 1

2. Machining center:

The Machining Center is a station used to manufacture lids and bases from raw materials. First, the articulated robot waits for raw material to be placed at the entry bay. When new material is detected, it is loaded into the CNC machine which will start manufacturing an item. Each item type takes a different interval of time to be produced (lids: 6 seconds; bases: 3 seconds). Once the operation is complete, the robot places the item on the exit bay.



Fig. 2

3. **Assembly unit:**

This part can be used to assemble Lids on Bases or pick and place items from one place to another. To guarantee a correct fit, bases and lids should be properly aligned by positioning bars.

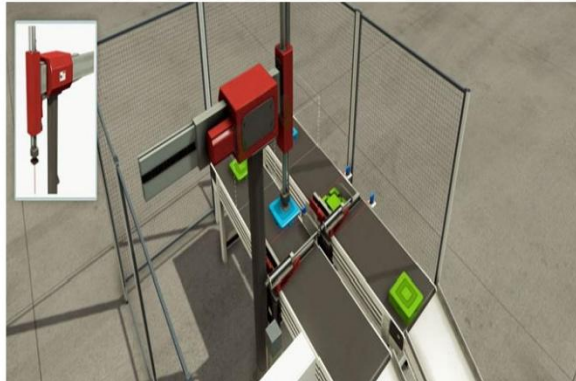


Fig. 3

Implementation Guidelines:

- a) Design a full production line whose operation is as follows:
 1. Generate random raw materials in the feeding station (Green or Blue).
 2. Move them using the basic pick and place robot into the output conveyor of the feeding unit.
 3. Use the machining center station (or multiples of them) to fabricate different products: blue lids, blue bases, green lids or green bases.
 4. Use the assembly station (or multiples of them) to assemble full green or blue products.

5. Sort the products into their designated removers (or storage areas) according to their colour.
6. Keep track of the finished products of each colour using a suitable interface.
7. Stop the production line when the given number of products of each colour is reached.
- b) Program the production line to achieve the required process control using Siemens TIA portal software.
- c) Design a suitable HMI interface on TIA portal to indicate the state of each station in the production line.
- d) Use analog configurations on belt drive of Feeding unit and Assembly unit
- e) Use analog and digital configurations on pick and place unit embedded in feeding and assembly units
- f) Control the speed of the conveyors according to the queue and the status of the machine center
- g) build a fault detection system for feeding and assembly unit and machine center
- h) Utilize field-related alarms for the detection of hardware and sequence faults.
- i) Prepare a technical report about the task highlighting your results and discussing your developed system performance. The report shall include the contribution of each team member in performing all task activities in a tabular form.

Note: Naturally, you will need to add more components to the mentioned stations (ex. indexing tables, vision sensors, etc.) in order to complete your line. Access to the description of all components in the FACTORY I/O library can be accessed through the following link:
<https://docs.factoryio.com/manual/parts/>

Deliverables:

1. Software TIA portal code
2. HMI interface on TIA portal
3. SIL/HIL simulation of the automated production line
4. A technical report of the task highlighting your results and discussing your developed system performance. The report shall include the contribution of each team member in performing all task activities in a tabular form.

Date of Submission: Week 14

Submitted Assessment Task



MCT 313 – Automation Milestone (1)

Submitted to:

Dr. Mohamed Omar

Eng. Mina Maher

Eng. Shahd Mohsen

Prepared by:

Kenzy Ahmed Abd El Moneam (22P0107)

Ahmed Mohamed Behiry (22P0044)

Rewan Mohamed (23P0383)

Contents

1. Introduction	4
2. System Overview	4
2.1 Selected System	4
2.2 Software Stack	4
2.3 Communication Architecture	5
3. TIA Portal Project Structure	5
3.1 Organization Blocks	5
3.2 PLC Tag Table	6
4. Signal Processing – Main_1 [OB123]	8
4.1 Network 1 – Level Meter Scaling	8
4.2 Network 2 – Flow Meter Scaling	8
4.3 Network 3 – Setpoint Scaling	9
4.4 Network 4 – Fill Valve Output Scaling	9
4.5 Network 5 – High Level Alarm	9
5. PID Control – Cyclic Interrupt OB	10
5.1 PID_Compact Block Configuration	10
5.2 PID Tuning Procedure	10
5.3 Final Tuned PID Parameters	11
6. Factory I/O SIL Simulation Setup	12
6.1 Scene Configuration	12
6.2 Driver Configuration	12
6.3 Tag Mapping	13
7. HMI Interface Design	14
7.1 HMI Device	14
7.2 HMI Tag Table	14
7.3 HMI Screen Layout	15
7.4 Alarm Configuration	16
8. System Testing and Results	17
8.1 Connection Verification	17
8.2 PID Performance	17
8.3 Alarm Testing	17

8.4 HMI Testing	17
9. Challenges and Solutions	18
10. Team Member Contributions	19
11. Conclusion	20
References.....	20

List of Figures

Figure 1-TIA Portal project structure	6
Figure 2- SIMATIC WinCC HMI main screen 1	8
Figure 3-PID_Compact block in OB30 (Cyclic Interrupt) and TIA Portal commissioning interface... ..	11
Figure 4- Factory I/O Level Control scene.....	12
Figure 5- SIMATIC WinCC HMI main screen2.....	16

1. Introduction

This report documents the design, implementation, and testing of an industrial liquid level control system developed as part of the MCT313 Automation course major task (Phase 1). The system uses a Siemens SIMATIC S7-1200 PLC simulated via S7-PLCSIM, programmed through TIA Portal V16, interfaced with Factory I/O for plant simulation, and monitored through a SIMATIC WinCC HMI.

The primary control objective is to maintain a desired water level in a tank by modulating a fill valve using a PID_Compact controller. The system includes signal scaling, real-time monitoring, operator control via HMI, and alarm management for fault detection.

2. System Overview

2.1 Selected System

The team selected Option A: Industrial Liquid Level System. The plant was simulated using the Level Control scene in Factory I/O, which includes a tank, a fill valve, a discharge valve, a level meter sensor, and a flow meter sensor.

2.2 Software Stack

Software	Version	Role
TIA Portal	V16	PLC programming, HMI design, PID configuration
S7-PLCSIM	V16	Virtual PLC simulation (S7-1200)
Factory I/O	V2.4.3	3D plant simulation and SIL interface
SIMATIC WinCC	V16 (Advanced)	HMI runtime and screen design
Npcap	1.87	Network loopback adapter for PLC-Factory I/O link

2.3 Communication Architecture

The communication between TIA Portal, S7-PLCSIM, and Factory I/O was established as follows:

- TIA Portal programs the virtual PLC via PROFINET over software loopback (IP: 192.168.0.1)
- Factory I/O connects to PLCSIM using the Siemens S7-PLCSIM driver
- The Factory I/O template project (FactoryIO_Template_S7-1200_V15_V16) was used as the base, which includes the MHJ-PLC-Lab-Function-S71200 [FC9000] communication block in Main [OB1]
- PUT/GET communication was enabled in the PLC protection settings to allow Factory I/O read/write access

3. TIA Portal Project Structure

3.1 Organization Blocks

OB	Name	Type	Purpose
OB1	Main	Program Cycle	Hosts FC9000 communication function (template). Not modified.
OB123	Main_1	Program Cycle	All signal scaling networks: level, flow, setpoint, fill valve output, and alarm logic
OB30	Cyclic Interrupt	Cyclic Interrupt (250ms)	Executes PID_Compact block at fixed sample interval



The following tags were defined in the Standard Variable Table:

6

Tag Name	Data Type	Address	Description
Setpoint	Real	%ID38	Raw setpoint input from potentiometer (0-10)
Normalized_SP	Real	%MD66	Setpoint normalized to 0.0-1.0
Scaled_SP	Real	%MD70	Setpoint scaled to 0-300 cm
PID_output	Real	%MD104	PID controller output (0-100%)
Normalized_Fill	Real	%MD74	PID output normalized to 0.0-1.0
Fill_Valve	Real	%QD30	Scaled fill valve command (0-10) to Factory I/O
PV_Display	DInt	%QD42	Integer PV value for Factory I/O display
SP_Display	DInt	%QD38	Integer SP value for Factory I/O display
Reset_PID	Bool	%M2.0	PID reset bit
Start_PB	Bool	%I0.0	Start pushbutton input
Stop_PB	Bool	%I0.1	Stop pushbutton input
Reset_PB	Bool	%I0.2	Reset pushbutton input
Discharge_Valve	Real	%QD34	Discharge valve output
High_Level_Alarm	Bool	%M10.0	Alarm bit: triggered when level exceeds 270 cm



Figure 2- SIMATIC WinCC HMI main screen 1

4. Signal Processing – Main_1 [OB123]

Main_1 contains all analog signal conditioning networks. Each sensor input follows a Normalize → Scale → Convert chain before being used by the PID controller or sent to displays.

4.1 Network 1 – Level Meter Scaling

Converts the raw level meter analog signal into engineering units for display and PID control.

- NORM_X: Input = Level_Meter (%ID30), MIN = 0.0, MAX = 10.0, Output = Normalized_Level (%MD50)
- SCALE_X: Input = Normalized_Level, MIN = 0.0, MAX = 300.0, Output = Scaled_Level (%MD54)
- CONV (Real to DInt): Input = Scaled_Level, Output = PV_Display (%QD42) for Factory I/O integer display

4.2 Network 2 – Flow Meter Scaling

- NORM_X: Input = Flow_Meter (%ID34), MIN = 0.0, MAX = 10.0, Output = Normalized_Flow (%MD58)

- SCALE_X: Input = Normalized_Flow, MIN = 0.0, MAX = 100.0, Output = Scaled_Flow (%MD62)

4.3 Network 3 – Setpoint Scaling

- NORM_X: Input = Setpoint (%ID38), MIN = 0.0, MAX = 10.0, Output = Normalized_SP (%MD66)
- SCALE_X: Input = Normalized_SP, MIN = 0.0, MAX = 300.0, Output = Scaled_SP (%MD70)
- CONV (Real to DInt): Input = Scaled_SP, Output = SP_Display (%QD38) for Factory I/O display

4.4 Network 4 – Fill Valve Output Scaling

Converts the PID controller output (0-100%) back into the valve command range expected by Factory I/O (0-10).

- NORM_X: Input = PID_output (%MD104), MIN = 0.0, MAX = 100.0, Output = Normalized_Fill (%MD74)
- SCALE_X: Input = Normalized_Fill, MIN = 0.0, MAX = 10.0, Output = Fill_Valve (%QD30)

4.5 Network 5 – High Level Alarm

A comparator network triggers the high level alarm when the tank level exceeds the safety threshold.

- CMP >=R: IN1 = Scaled_Level (%MD54), IN2 = 270.0
- Output coil: High_Level_Alarm (%M10.0) — activates when level reaches 90% of full scale

5. PID Control – Cyclic Interrupt OB

5.1 PID_Compact Block Configuration

The PID_Compact block was placed in the Cyclic Interrupt OB with a sample time of 250,000 microseconds (250ms). This ensures the controller executes at a fixed, constant time interval independent of the main program scan cycle.

Parameter	Value
Controller type	Length (cm) — Automatic mode
Setpoint source	Scaled_SP (%MD70) — range 0 to 300 cm
Process value source	Scaled_Level (%MD54) — range 0 to 300 cm
Output	PID_output (%MD104) — range 0 to 100%
Reset input	Reset_PID (%M2.0)
Process value limits	Low: 0.0 cm, High: 300.0 cm
Controller structure	PI (derivative term disabled for slow level dynamics)
Cyclic interrupt time	250,000 μ s (250 ms)

5.2 PID Tuning Procedure

The PID_Compact block was tuned using the built-in TIA Portal commissioning tool:

- Step 1 — Reset_PID was pulsed (0→1→0) to clear any previous controller state
- Step 2 — Pretuning mode was selected and started. The controller performed an automatic step test and estimated initial PI parameters
- Step 3 — The system was observed in Factory I/O as the level converged toward the setpoint

- Step 4 — Fine tuning mode was applied to reduce overshoot and improve settling time
- Step 5 — Final parameters were uploaded and saved to the project (Download to device) to prevent loss when going offline

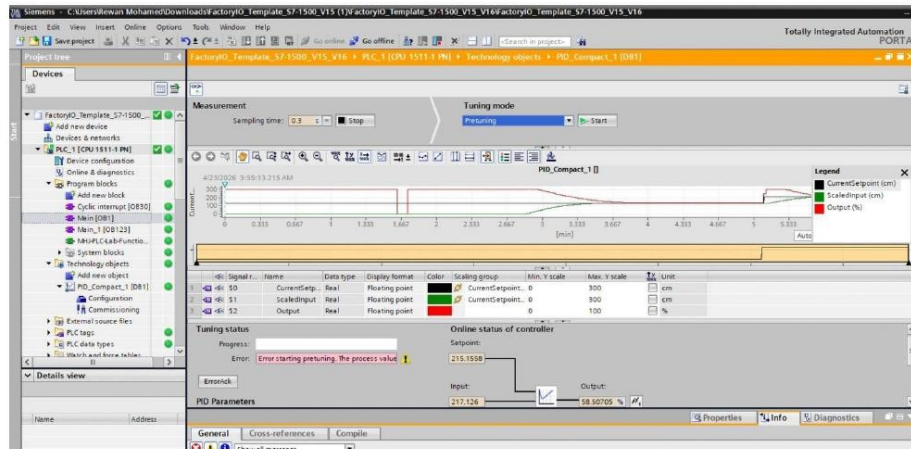
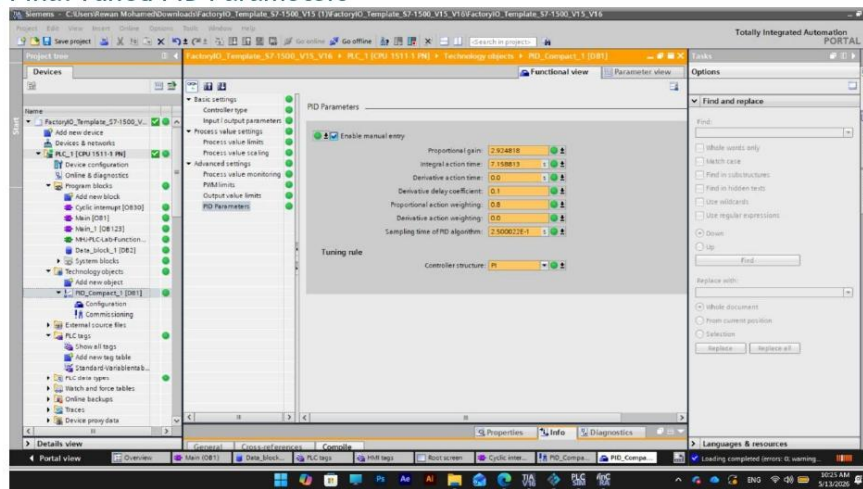


Figure 3-PID_Compact block in OB30 (Cyclic Interrupt) and TIA Portal commissioning interface.

5.3 Final Tuned PID Parameters



6. Factory I/O SIL Simulation Setup

6.1 Scene Configuration

The Level Control scene was loaded in Factory I/O. The scene includes a tank, fill valve, discharge valve, level meter, flow meter, setpoint potentiometer, and SP/PV displays.

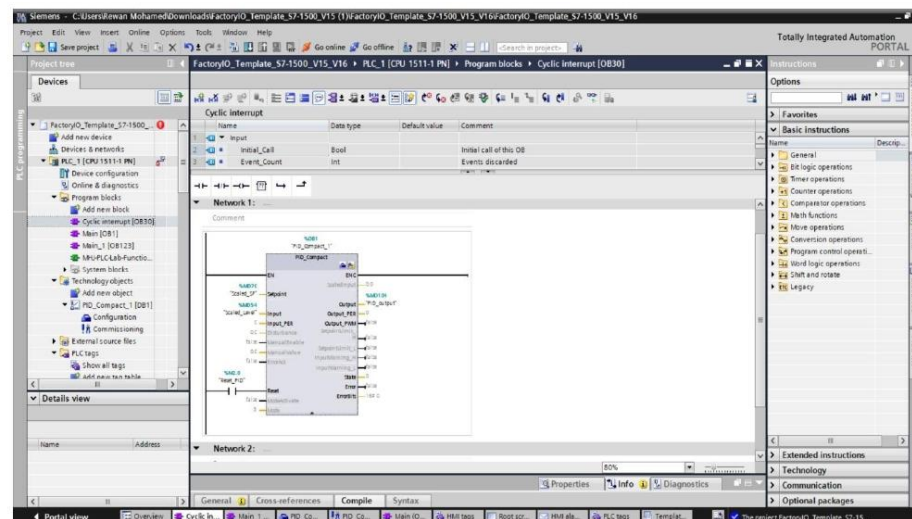


Figure 4- Factory I/O Level Control scene

6.2 Driver Configuration

- Driver selected: Siemens S7-PLCSIM
- This driver communicates directly with S7-PLCSIM V16 running on the same machine
- Connection confirmed by green indicator in Factory I/O driver panel
- Tag mapping was verified through the Factory I/O driver window showing correct I/O addresses

6.3 Tag Mapping

Factory I/O Tag	Direction	PLC Address	Description
Level meter	Sensor → PLC	%ID30	Tank level analog reading
Flow meter	Sensor → PLC	%ID34	Flow rate analog reading
Setpoint	Sensor → PLC	%ID38	Operator setpoint from potentiometer
Start	Sensor → PLC	%I0.0	Start pushbutton
Stop	Sensor → PLC	%I0.1	Stop pushbutton
Reset	Sensor → PLC	%I0.2	Reset pushbutton
Fill valve	PLC → Actuator	%QD30	Fill valve opening command
Discharge valve	PLC → Actuator	%QD34	Discharge valve command
PV display	PLC → Actuator	%QD42	Integer level value for panel display
SP display	PLC → Actuator	%QD38	Integer setpoint value for panel display

7. HMI Interface Design

7.1 HMI Device

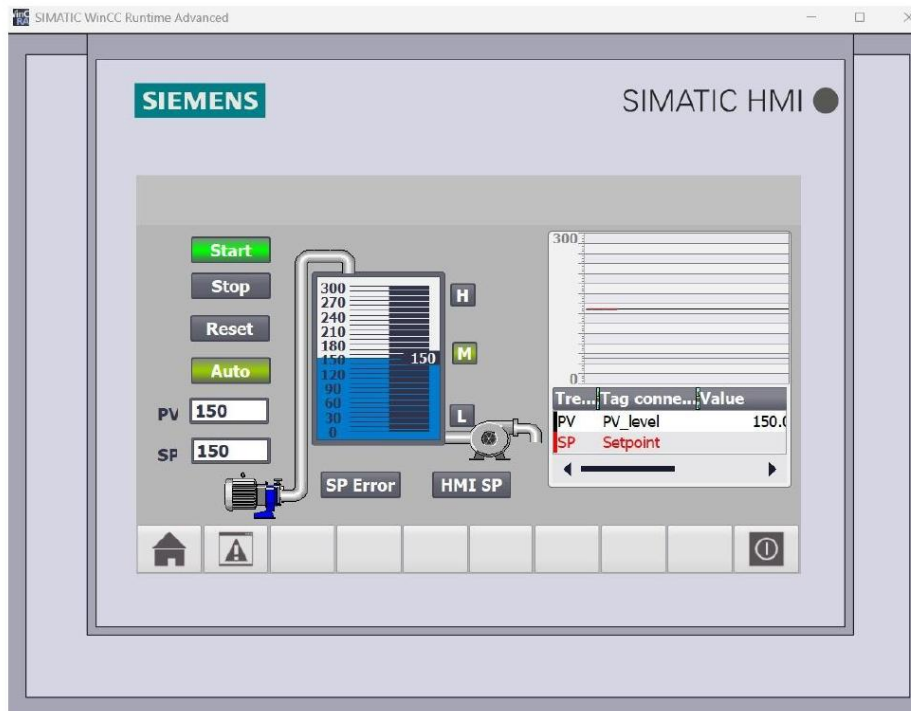
An HMI panel was added to the TIA Portal project using WinCC Advanced V16. The panel was configured with a PROFINET connection to PLC_1 sharing the same subnet as the simulated controller.

7.2 HMI Tag Table

HMI Tag	PLC Tag	Address	Data Type	Purpose
Level_PV	Scaled_Level	%MD54	Real	Displays current tank level in cm
Setpoint_SP	Scaled_SP	%MD70	Real	Displays and sets desired level
Flow_reading	Scaled_Flow	%MD62	Real	Displays current flow rate
Start_button	Start_PB	%I0.0	Bool	Starts the control system
Stop_button	Stop_PB	%I0.1	Bool	Stops the control system
Reset_button	Reset_PID	%M2.0	Bool	Resets the PID controller
Running_status	Start_Light	%Q0.0	Bool	System running indicator
High_Level_Alarm	High_Level_Alarm	%M10.0	Bool	High level alarm status

7.3 HMI Screen Layout

The main HMI screen (Root screen) contains the following elements:



Element	Type	Tag Linked	Function
Tank level bar	Bar graph	Level_PV	Animated bar showing current level (0-300 cm)
Current level display	IO Field (Output)	Level_PV	Numeric display of current level in cm
Setpoint display	IO Field (Input/Output)	Setpoint_SP	Operator can read and write setpoint

Flow rate display	IO Field (Output)	Flow_reading	Numeric display of flow rate
START button	Button	Start_button	SetBitWhileKeyPressed — activates system
STOP button	Button	Stop_button	SetBitWhileKeyPressed — stops system
RESET button	Button	Reset_button	SetBitWhileKeyPressed — resets PID
Status lamp	Circle	Running_status	Green = running, Red = stopped
Alarm display	Alarm display	Automatic	Shows active alarms from HMI alarm list



Figure 5- SIMATIC WinCC HMI main screen2

7.4 Alarm Configuration

One discrete alarm was configured in the HMI alarms section:

Alarm Name	Alarm Text	Trigger Tag	Trigger Condition	Alarm Class
High Level Alarm	WARNING: Tank level too high!	High_Level_Alarm (%M10.0)	Bit = 1 (level > 270 cm)	Errors

The alarm is triggered by the CMP $\geq$ R compare block in Main_1 OB123 when Scaled_Level exceeds 270.0 cm (90% of full scale). The alarm display element on the HMI screen automatically shows the alarm text when the condition is active.

8. System Testing and Results

8.1 Connection Verification

- TIA Portal successfully compiled and downloaded to S7-PLCSIM V16 with 0 errors
- Factory I/O connected to PLCSIM via Siemens S7-PLCSIM driver with green connection indicator
- HMI simulation launched successfully via WinCC Runtime

8.2 PID Performance

- Setpoint tracking: The level successfully follows the desired setpoint after tuning
- Pretuning provided initial PI parameters which were further refined using fine tuning
- PI structure was selected (no derivative) as level systems have slow dynamics and the derivative term is not necessary
- The controller output (fill valve) modulates smoothly between 0% and 100% without chattering

8.3 Alarm Testing

- High level alarm activates correctly when Scaled_Level exceeds 270 cm
- Alarm text appears in the HMI alarm display element
- Alarm clears automatically when level drops below the threshold

8.4 HMI Testing

- START button activates the control system and status lamp turns green
- STOP button deactivates the system and status lamp turns red
- RESET button pulses Reset_PID tag to reinitialize the PID controller
- Setpoint IO field allows operator to enter new desired level values
- Bar graph animates correctly showing real-time tank level

6. Challenges and Solutions

Challenge	Root Cause	Solution
S7-PLCSIM not installed	TIA Portal V16 installed without PLCSIM component	Downloaded and installed SIMATIC S7-PLCSIM V16 trial from Siemens support portal
Factory I/O connection failure	Wrong driver selected (S7-1200/1500 Ethernet instead of S7-PLCSIM)	Switched to Siemens S7-PLCSIM driver which communicates directly with running PLCSIM instance
Timer block compilation errors	TOF V1.0 blocks copied from S7-300 project are incompatible with S7-1200	Deleted old timer DBs and re-added fresh TOF blocks from Instructions panel
Wrong CPU selected initially	Project started with S7-300 CPU which PLCSIM V16 does not support	Created new project using CPU 1212C DC/DC/DC (S7-1200 series)
PUT/GET access denied	Protection setting not enabled by default	Enabled in Device configuration > Protection & Security > Connection mechanisms
Tag address conflicts	Template project had pre-existing Tag_1/2/3 occupying needed addresses	Deleted template placeholder tags and reassigned all addresses correctly

10. Team Member Contributions

Activity	Rewan	Ahmed	Kenzy
TIA Portal project setup and template configuration	✓		
PLC tag table definition	✓		
Main_1 OB123 scaling networks (NORM/SCALE)		✓	
Cyclic Interrupt OB and PID_Compact block		✓	
PID tuning (pretuning and fine tuning)		✓	
Factory I/O scene setup and driver configuration	✓		
HMI device addition and PLC connection	✓		
HMI tag table and screen design	✓		
Alarm configuration and testing	✓		
System integration testing	✓	✓	✓
Technical report preparation	✓		✓

11. Conclusion

This project successfully demonstrated the implementation of a complete industrial PLC-based liquid level control system. The system integrates TIA Portal programming, S7-PLCSIM virtual PLC simulation, Factory I/O plant simulation for SIL testing, and a SIMATIC WinCC HMI for operator interaction.

The PID_Compact controller in the Cyclic Interrupt OB achieved stable level control with good setpoint tracking after applying the two-stage tuning procedure (pretuning followed by fine tuning). The signal scaling chain correctly converts all analog values between Factory I/O raw ranges and engineering units. The HMI provides operators with real-time visualization, setpoint adjustment capability, and alarm notification for high level conditions.

Several technical challenges were encountered and resolved during implementation, particularly regarding software compatibility between TIA Portal V16, S7-PLCSIM V16, and Factory I/O. These were systematically debugged and resolved, demonstrating practical troubleshooting skills relevant to industrial automation engineering.

References

- Factory I/O Level Control Scene Documentation:
<https://docs.factoryio.com/manual/scenes/level-control/>
- Siemens PID Control with PID_Compact for S7-1200/S7-1500:
support.industry.siemens.com, Document 100746401
- S7-1500 PID Control Function Manual: support.industry.siemens.com
- Factory I/O S7-PLCSIM Driver Documentation:
<https://docs.factoryio.com/manual/drivers/s7-plcsim/>
- MCT313 Lab 3 — Introduction to TIA Portal, Factory I/O and HMI, Spring 2026
- MCT313 Lab 4 — Tank Level PID Control Using TIA Portal and Factory I/O, Spring 2026



MCT 313 – Automation Phase 2

Submitted to: Dr. Mohamed Omar Eng. Mina Maher Eng. Shahd Mohsen

Prepared by:

Kenzy Ahmed Abd El Moneam (22P0107)

Ahmed Mohamed Behiry (22P0044)

Rewan Mohamed (23P0383)

Table of Contents	2
1. Introduction	4
2. Task Requirements	4
2.1 Design and Simulation	4
2.2 Programming and Control	4
2.3 HMI Development	5
2.4 Fault Detection and Alarms	5
3. System Overview and Material Flow	5
3.1 Material Flow	5
3.2 Stations and Responsibilities	5
4. I/O Configuration	7
4.1 Machining Centre Inputs	7
4.2 Machining Centre Outputs	7
4.3 Sorting and Alignment	8
5. Software Architecture	9
5.1 Code Blocks	9
5.2 Global Data Blocks	9
6. Sorting Station Implementation	11
6.1 Vision Sensor Configuration	11
6.2 State Machine	11
6.3 Alternating Routing	11
6.4 Belt Gating and Edge Detection	11
7. Alignment Pushers	12
8. Machining Centre Subsystem	13
8.1 State Machine Overview	13
8.2 Critical Implementation Details	13
8.2.1 Loading Delay	13
8.2.2 Machining Transition Condition	13
8.2.3 Drain State	13
8.2.4 Timeout	14
8.3 Exit Conveyor Strategy and Merge Coordination	14
8.3.1 Queue Diffuse Sensors	14
8.3.2 Priority and Gating	14
8.3.3 Exit Conveyor Assignment Outside the CASE Block	14
9. Assembly Station Integration	16
9.1 Assembly Subsystem	16

9.2 The Run Wiring Fix	16
9.3 Removing the Test Emitter	16
10. Feeding Station and Cross-Station Gating	16
10.1 Feeding Architecture	16
10.2 Stopping the Feeding When Machines Are Saturated	17
10.3 Hold on Output Belt When Destination Unavailable	17
11. Fault Detection and Handling	18
12. Supervisory Control and HMI	18
13. Testing and Validation	18
14. Challenges and Solutions	19
15. Further Development	19
16. Conclusion	20
17. Drive Link	20

1. Introduction

Industrial automation is the backbone of modern manufacturing, replacing repetitive manual operations with programmable, sensor-driven control systems that deliver higher throughput, consistent quality, and traceable production. Programmable Logic Controllers (PLCs) sit at the centre of this transformation: they coordinate motors, conveyors, robotic arms, and machining centres according to programmed logic that can be modified in software rather than rewired in hardware.

This report documents Phase 2 of the MCT313 Major Task: the design, programming, simulation, and integration of a complete production line built around two machining centres. The system feeds raw material from an emitter, sorts parts by colour and routes them to the correct CNC machine, machines lids and bases in parallel, transports finished parts on a coordinated exit conveyor, and finally pushes them into an assembly station where two robotic arms mate lids to bases to produce the final products.

The work was implemented on Siemens TIA Portal V16 driving a simulated SIMATIC S7-1500 (CPU 1511-1 PN) via S7-PLCSIM, with the physical scene modelled in Factory I/O. The deliverable is a fully integrated production line in which the feeding, sorting, machining, and assembly stations communicate through a shared Data Block architecture and operate as a single coordinated machine.

This report focuses primarily on the machining centre subsystem (the author's individual responsibility) and the system-level integration work that ties feeding, sorting, machining, and assembly together. The control panel, HMI, and detailed fault-detection strategies referenced in the Phase 1 plan are summarised at the relevant points.

2. Task Requirements

The Phase 2 brief expanded on the Phase 1 design with concrete deliverables across four areas:

2.1 Design and Simulation

- Design a fully functional production line in Factory I/O incorporating a feeding unit, sorting mechanism, two machining centres (one for lids, one for bases), and an assembly station.
- Use the minimum set of sensors and actuators required to achieve the operation, optimising for cost and signal economy without sacrificing throughput.

2.2 Programming and Control

- Program the system in Siemens TIA Portal V16 using a combination of Structured Control Language (SCL) and Ladder Diagram (LAD).
- Implement state-machine logic for each station with clear, named states and well-defined transitions.
- Coordinate conveyor speed and gating so parts neither pile up nor fall from the line.

- Provide hand-off variables between stations so the line behaves as a single integrated machine rather than four independent subsystems.

2.3 HMI Development

- Provide a supervisory interface to monitor station state, current production counts, and alarm conditions.
- Allow the operator to set production targets and acknowledge faults.

2.4 Fault Detection and Alarms

- Detect simulator faults exposed by Factory I/O (such as the CNC "has error" output) and react safely.
- Add sequence-level alarms (machining timeout, pusher not returned, robot lost item) and present them to the operator.

3. System Overview and Material Flow

The production line is organised into four functional stations connected by a single material flow path. Understanding the flow is essential before discussing the individual code blocks, because every control decision in the machining and exit logic is driven by what is happening upstream and downstream.

3.1 Material Flow

Raw materials emerge from an emitter at the feeding station as blue or green raw cylinders, distributed randomly. A two-axis pick-and-place robot transfers each part from the feeding belt to the output belt, where it travels past a colour vision sensor and into a pop-up wheel sorter. The sorter routes parts left or right onto one of two parallel raw conveyors (Belt 4 to the lids machine, Belt 5 to the bases machine). Before each machining centre an alignment pusher squares the part on the conveyor so the CNC's pick-up routine can grip it cleanly.

After machining, each centre releases its finished part onto a two-segment exit conveyor (ExitConv1 and ExitConv2). Both lanes merge onto a shared main exit conveyor which feeds the assembly station. At the assembly station, four pushers sort the arriving stream by colour and type into four lanes (green base, green lid, blue base, blue lid). A clamp holds each part while a robotic arm picks up the lid, traverses to the base lane, and places the lid on the base to form the final product.

3.2 Stations and Responsibilities

Station	Purpose	Key Components
Feeding	Generate raw materials and load them onto the output conveyor.	Emitter, feeding belt, pick-and-place robot, output belt, drop sensor.

Sorting	Identify colour and route parts to the correct machining centre.	Two colour vision sensors, pop-up wheel sorter, Belt 4 (to lids), Belt 5 (to bases).
Alignment	Square the part on the conveyor before entry to the CNC.	Two pneumatic pushers with front and back limit sensors, two presence vision sensors.
Machining (MC1, MC2)	Machine raw materials into lids (MC1) or bases (MC2).	CNC machine, raw conveyor, exit conveyor (two segments), entry and exit sensors, busy sensor.
Exit / Merge	Transport finished parts toward assembly with no collisions.	ExitConv1 and ExitConv2 per machine, shared main exit conveyor, queue diffuse sensors.
Assembly	Sort by colour, clamp, and assemble lids onto bases.	Four pushers, four clamps, two robotic arms, sorting belts, positioners, counter.

4. I/O Configuration

The I/O map below shows the addresses used throughout the machining and integration code. Inputs and outputs are organised in PLC tag tables grouped by station so that responsibility for each signal is unambiguous when several students collaborate on the project.

4.1 Machining Centre Inputs

Tag	Address	Description
MC1_Lids_AtEntry	%I0.0	Entry sensor at lids CNC (NC: TRUE = empty).
MC1_Lids_Busy	%I0.1	Lids CNC busy signal (NO: TRUE = machining).
MC1_Lids_HasError	%I0.2	Lids CNC simulator fault output.
MC1_Lids_AtExit	%I0.3	Exit sensor at lids CNC (NC: TRUE = empty).
MC2_Bases_AtEntry	%I0.4	Entry sensor at bases CNC.
MC2_Bases_Busy	%I0.5	Bases CNC busy signal.
MC2_Bases_HasError	%I0.6	Bases CNC simulator fault output.
MC2_Bases_AtExit	%I0.7	Exit sensor at bases CNC.
Sensor_LidsExit_Queue	%I1.1	Diffuse sensor on Lids exit conveyor 2 (queue detection).
Sensor_BasesExit_Queue	%I1.2	Diffuse sensor on Bases exit conveyor 2 (queue detection).

4.2 Machining Centre Outputs

Tag	Address	Description
MC1_Lids_RawConveyor	%Q0.0	Raw conveyor feeding lids CNC.
MC1_Lids_Produce	%Q0.1	Command lids CNC to produce a part.
MC1_Lids_Start	%Q0.2	Start lids CNC cycle.
MC1_Lids_Reset	%Q0.3	Reset lids CNC after fault.
MC1_Lids_Stop	%Q0.4	Stop lids CNC.
MC1_Lids_ExitConv1	%Q0.5	Lids exit conveyor segment 1.
MC1_Lids_ExitConv2	%Q0.6	Lids exit conveyor segment 2.
MC2_Bases_RawConveyor	%Q0.7	Raw conveyor feeding bases CNC.
MC2_Bases_Produce	%Q1.0	Command bases CNC to produce a part.
MC2_Bases_Start	%Q1.1	Start bases CNC cycle.
MC2_Bases_ExitConv1	%Q1.4	Bases exit conveyor segment 1.
MC2_Bases_ExitConv2	%Q1.5	Bases exit conveyor segment 2.

4.3 Sorting and Alignment

Tag	Address	Description
Vision_Blue	%I3.7	Vision sensor 1: TRUE when blue raw material detected.
Vision_Green	%I5.0	Vision sensor 2: TRUE when green raw material detected.
Sort_SorterOn	%Q5.1	Pop-up wheel sorter activation.
Sort_Left	%Q5.2	Sorter direction left (to lids).
Sort_Right	%Q5.3	Sorter direction right (to bases).
Belt_ToLids	%Q5.4	Belt conveyor 4 carrying parts toward MC1.
Belt_ToBases	%Q5.5	Belt conveyor 5 carrying parts toward MC2.
Sensor_Align_Lids_ID	%ID50	All-ID vision sensor in front of Pusher 1.
Sensor_Align_Bases_ID	%ID54	All-ID vision sensor in front of Pusher 2.
Pusher_1_On	%Q6.7	Alignment pusher 1 (lids lane).
Pusher_1_FrontLimit	%I9.4	Pusher 1 fully extended.
Pusher_1_BackLimit	%I9.5	Pusher 1 at home position.
Pusher_2_On	%Q7.0	Alignment pusher 2 (bases lane).
Pusher_2_FrontLimit	%I9.6	Pusher 2 fully extended.
Pusher_2_BackLimit	%I9.7	Pusher 2 at home position.

5. Software Architecture

The program is organised around a centralised OB1 that calls one function per station and three global Data Blocks that hold all shared state. This separation makes responsibility clear: each FC owns the actuators of its own station, and every cross-station hand-off goes through a named DB variable rather than a direct read of a foreign FC's internal logic.

5.1 Code Blocks

Block	Type	Responsibility
OB1 (Main)	Cyclic OB	Calls each station function in order: feeding → sorting → alignment → lids → bases → assembly. Also aggregates Fault_Active.
FB_Feeding	Function Block	Emitter, feeding belt, pick-and-place sequence, output belt timing, part counters.
FC_Sorting	Function	Vision sensor edge detection, alternating direction logic, pop-up sorter control, Belt 4 and Belt 5 gating.
FC_Align_Pushers	Function	Two alignment pushers in front of each machining centre.
FC_Lids (MC1)	Function	Lids machining state machine and exit conveyor control.
FC_Bases (MC2)	Function	Bases machining state machine and exit conveyor control.
FB_Assembly_Standalone	Function Block	Four assembly pushers, four lanes with clamps and conveyors, two assembly robots, main exit conveyor interlock, assembly fault detection.

5.2 Global Data Blocks

Three global Data Blocks hold the system's shared state. Keeping them separate makes the architecture more modular than a single monolithic DB.

DB_Machining holds state for the two machining centres:

```
Lids_State, Bases_State      : Int      (0=IDLE 1=LOADING 2=PRODUCE 3=MACHINING
                                         4=TRANSPORT 5=DRAIN 6=COMPLETE 9=FAULT)
Lids_Count, Bases_Count     : DInt
Target_Lids, Target_Bases   : DInt
Lids_Timeout_ET, Lids_LoadDelay_ET, Bases_Timeout_ET, Bases_LoadDelay_ET : Time
HMI_Start, HMI_Stop, HMI_Reset, HMI_FaultAck      : Bool
Fault_Lids_Sim, Fault_Lids_Timeout, Fault_Bases_Sim, Fault_Bases_Timeout : Bool
Fault_Active, Production_Done, MC_LidReady, MC_BaseReady : Bool
```

DB_Sorting holds the sorting state machine and routing flags:

```
Sort_State           : Int (0=IDLE 1=SORT 2=CLEAR)
Blue_Count, Green_Count : Int
Last_Was_Blue       : Bool
Sort_Active          : Bool
Sort_Timer_ET        : Time
Part_GoingToLids, Part_GoingToBases : Bool
```

System_Armed (%M20.0) is set TRUE by the control-panel network when the operator presses Start and reset by Stop, E-Stop, or any latched fault. Every station gates its actuators on this flag.

6. Sorting Station Implementation

The sorting station is the first cross-station integration point: it takes the random colour stream from the feeder and balances the load between the two machining centres so neither is starved nor flooded.

6.1 Vision Sensor Configuration

Factory I/O's vision sensor was originally configured with the "Blue Base" detection option, which proved unsuitable: a single Bool output cannot distinguish "green part present" from "no part present." After testing the "All Digital" and "All ID" options, the cleanest solution was to install two vision sensors at the same position above the conveyor, each tuned to a single colour. Vision Sensor 1 raises TRUE only on Blue Raw Material; Vision Sensor 2 only on Green Raw Material. The OR of the two signals serves as a generic "part present" trigger, and the individual signals identify the colour.

6.2 State Machine

The sorting logic is a three-state machine driven by rising-edge detection on the two vision signals:

- State 0 (IDLE): Sorter and direction outputs are FALSE. A rising edge on either Vision_Blue or Vision_Green latches the colour into Last_Was_Blue and advances to state 1.
- State 1 (SORT): The state machine consults the colour counter and sets the direction (Sort_Left or Sort_Right) and the routing flag (Part_GoingToLids or Part_GoingToBases), then activates Sort_SorterOn and moves to state 2.
- State 2 (CLEAR): The sorter is held active for 2.5 seconds (the time required for the part to pass the wheel), then the counter for the relevant colour is incremented (with modulo-2 reset) and the state returns to 0.

6.3 Alternating Routing

The reference design from the Olives team used independent counters for each colour and routed every other part of the same colour to the opposite machine. The same approach is implemented here: the first blue goes to the lids machine, the second blue to the bases machine, the third blue back to the lids, and so on, with green following its own independent rotation. This guarantees that both MC1 and MC2 always receive a balanced load and that the assembly station eventually sees an equal number of lids and bases in each colour.

6.4 Belt Gating and Edge Detection

Two implementation details were essential. First, the part is only under the vision sensor for a short interval; using the raw signal as a state-0 condition risks missing the event when the FC runs less frequently than the part transit time. R_TRIG rising-edge instances on each colour signal solved this by latching the transition rather than relying on a level read.

Second, Belt 4 (to lids) and Belt 5 (to bases) must not run when their destination machine cannot accept a part. The current implementation ties each belt directly to its raw conveyor signal (Belt_ToLids := MC1_Lids_RawConveyor and similarly for bases), and adds an extra check on the alignment sensor: if a part is already at the aligner, the belt stops so the next part does not collide with it.

7. Alignment Pushers

Between Belt 4/5 and the machining centre raw conveyor, parts often arrived rotated or off-centre. A small pneumatic pusher in each lane squares the part against a guide rail before the CNC's pick-up routine takes over.

The alignment logic is intentionally minimal — a two-condition latch rather than a full state machine:

```
IF Sensor_Align_Lids_ID > 0 AND Pusher_1_BackLimit THEN
    Pusher_1_On := TRUE;
ELSIF Pusher_1_FrontLimit THEN
    Pusher_1_On := FALSE;
END_IF;
```

Activation requires both that the All-ID vision sensor sees a part (any ID greater than zero) and that the pusher is at its back limit. The pusher releases as soon as it reaches the front limit, and a pneumatic return-stroke brings it back to the home position automatically.

8. Machining Centre Subsystem

The machining centre is the author's primary individual deliverable. The two centres run identical state machines (FC_Lids on MC1 and FC_Bases on MC2) parametrised on a different set of I/O tags. Both follow the same seven-state cycle plus a fault state.

8.1 State Machine Overview

State	Name	What Happens
0	IDLE	Raw conveyor ON; Produce/Start OFF; exit conveyors handled by global gating; wait for part at entry sensor.
1	LOADING	Raw conveyor stays ON for 2.5 s so the part fully enters the cage.
2	PRODUCE	Raw conveyor OFF; Produce and Start commanded; wait for Busy to confirm the CNC has accepted the part.
3	MACHINING	Hold Produce; Start released; arm timeout timer; advance when AtExit goes FALSE indicating the part has been placed on the exit.
4	TRANSPORT	MC_xReady flag set high; increment count when AtExit clears.
5	DRAIN	Continue running exits for 2 s so the back edge of the part fully clears the exit conveyor before the conveyor stops.
6	COMPLETE	Check target; if reached set Production_Done; return to state 0.
9	FAULT	Halt all motion, raise Reset and Stop on the CNC, wait for HMI_FaultAck.

8.2 Critical Implementation Details

Several details were the result of iterative debugging in Factory I/O and deserve a record so the same problems are not repeated by future students.

8.2.1 Loading Delay

The entry sensor (AtEntry) triggers as the front edge of the part crosses the beam. If the raw conveyor stops at that instant, the part comes to rest with the bulk of its body outside the CNC's pickup zone, and the machine never starts. A 2.5-second LoadDelay timer in state 1 keeps the conveyor running long enough for the part body to fully enter the cage.

8.2.2 Machining Transition Condition

State 3 advances on NOT AtExit (a part has appeared at the exit station inside the cage). The earlier version of this code also required NOT Busy, but the simulator sometimes held Busy high while the robot returned to its home position, even after the part had been placed. Using AtExit alone is both sufficient and more reliable: the appearance of the part at the exit is unambiguous evidence that machining has completed.

8.2.3 Drain State

Without state 5, the moment AtExit goes back to TRUE (back edge of part has crossed the exit sensor) the conveyors stop. Because there is a physical gap between the sensor beam and the end of the exit conveyor, the part is left half on, half off, blocking the next cycle. The 2-second drain state runs the exit conveyors for a fixed period after the part-clearing event so the part is fully transported off the machine before everything resets.

8.2.4 Timeout

State 3 includes a TON timer (60 s for lids, 45 s for bases) that latches Fault_Lids_Timeout or Fault_Bases_Timeout if machining does not complete. This catches simulator hangs and a few real Factory I/O scenarios where the CNC fails to receive the Produce command properly.

8.3 Exit Conveyor Strategy and Merge Coordination

This was the most architecturally interesting part of the integration work. The two machining centres dump finished parts onto a shared main exit conveyor that feeds the assembly station. If both centres finish at the same time, two parts arrive at the merge point simultaneously and collide; if one centre finishes first, the second part can still pile up behind it on the main belt.

8.3.1 Queue Diffuse Sensors

Two diffuse sensors were added to the scene at the downstream end of each exit conveyor: Sensor_LidsExit_Queue (%I1.1) and Sensor_BasesExit_Queue (%I1.2). They are NO sensors — TRUE means a part is currently passing over the sensor on its way to the merge.

8.3.2 Priority and Gating

After several design iterations the final policy is:

- The exit conveyors run by default whenever the system is armed. Putting the assignment at the top of the FC (outside the CASE block) means the belt is always TRUE every scan unless explicitly overridden.
- Bases has priority. The bases exit conveyors run unconditionally whenever System_Armed is high.
- Lids exit conveyors run unless Sensor_BasesExit_Queue is TRUE, in which case they stop. This guarantees that if a base is travelling on the merge, the lid waits, and the merge sees only one part at a time.
- The main exit conveyor (Conveyor_Exit_Main) is owned entirely by the assembly FB. The machining FCs do not touch it. The assembly FB drops the main belt to FALSE while any of its four pushers is active so that parts arriving at a pusher are not carried past it.

Together, these three rules prevent the merge collision, the pile-up on the main belt, and any duelling-master conflict over a single conveyor output.

8.3.3 Exit Conveyor Assignment Outside the CASE Block

Moving the exit-conveyor assignments out of the CASE block was a deliberate architectural choice. With the assignments inside individual states, every code change in the state machine risked breaking the conveyor behaviour. Lifting them to the top of the FC turns the conveyors into

background services driven by global flags (System_Armed, the queue sensors, the state-9 fault), and the state machine focuses on what actually changes per state: CNC commands, counters, and timer logic.

9. Assembly Station Integration

The assembly station was developed in collaboration with the second team member. The author's responsibility was to integrate it with the machining output so that parts machined by MC1 and MC2 are delivered cleanly to the four assembly lanes.

9.1 Assembly Subsystem

The assembly FB exposes a single Run input and contains four logically parallel subsystems:

- Four pusher state machines (Pusher 1 - green base, Pusher 2 - green lid, Pusher 3 - blue base, Pusher 4 - blue lid). Each pusher fires when its colour-and-type vision sensor sees a part, then returns to its back limit.
- Four lane subsystems, each with a presence sensor, a clamp, a clamp limit sensor, and a sorting belt. The clamp latches when the part arrives and unlocks during ejection.
- Two assembly robots (Green and Blue), each a twelve-state pick-and-place sequence that grabs the lid, traverses to the base lane, places the lid, and returns to home.
- A main-belt interlock that stops the main exit conveyor whenever any pusher is mid-stroke.
- A fault module that catches dropped items mid-traverse and pushers that fail to return within 5 s.

9.2 The Run Wiring Fix

During integration, the assembly station did not start: the main exit conveyor remained off, and the lane belts never moved. Investigation showed that the Run input on FB_Assembly_Standalone in OB1 was wired to a memory bit named AlwaysTRUE which was never actually set true. Re-wiring Run to System_Armed (%M20.0) — the master run flag shared with feeding, sorting, and machining — fixed the issue immediately and brought the assembly under the same control as the rest of the line.

9.3 Removing the Test Emitter

The assembly FB included a Test_Emitter actuator that injected dummy parts directly into the assembly conveyor for unit testing. After integration this was no longer needed because the real machined parts arrive from the machining centres. The emitter was removed from the Factory I/O scene and Test_Emitter was left in the code as an unused output (writing to a deleted actuator has no effect).

10. Feeding Station and Cross-Station Gating

10.1 Feeding Architecture

The feeding FB is a seven-step pick-and-place sequence built on a hybrid event-plus-timer architecture. Each axis movement is gated on the simulator's MovingX and MovingZ flags so that the FB advances only after the physical axis has actually stopped, with short settle delays to

remove tumbling artefacts. Counters track parts in and parts out, and the values are written to Factory I/O's digital displays.

10.2 Stopping the Feeding When Machines Are Saturated

When both machining centres are simultaneously machining (state 2 or 3), feeding new parts onto Belt 6 only causes them to pile up at the sorter. The feeding step 0 was extended with an override:

```
IF (Lids_State BETWEEN 2 AND 3) AND (Bases_State BETWEEN 2 AND 3) THEN
    Belt_Feeding := FALSE;
    Emitter      := FALSE;
ELSE
    Belt_Feeding := #Run;
    Emitter      := #Run;
END_IF;
```

10.3 Hold on Output Belt When Destination Unavailable

Initially Belt_Output ran on a 10-second TOF after Sensor_OutputDrop. But this could drop a part into the sorter even when the destination belt was off, causing the new part to collide with the part still waiting in front of the aligner. An override after the TOF gates the output belt off when the alignment sensors see a queued part:

```
IF Sensor_Align_Lids_ID > 0 OR Sensor_Align_Bases_ID > 0 THEN
    Belt_Output := FALSE;
END_IF;
```

This downgrade-friendly approach — let the TOF run by default and only stop the belt when needed — was preferred over a pure stop-and-start interlock because the existing TOF logic is conservative and reliable.

11. Fault Detection and Handling

Faults are aggregated into a single DB_Machining.Fault_Active flag which the OB1 raises high whenever any individual fault is latched. State-0 transition conditions in every station check NOT Fault_Active so the line refuses to start fresh cycles while a fault is open.

Fault	Source	Detection / Response
Lids_Sim	MC1_Lids_HasError	Mirrored each scan; forces Lids_State := 9 (FAULT).
Bases_Sim	MC2_Bases_HasError	Mirrored each scan; forces Bases_State := 9.
Lids_Timeout	60 s TON in state 3	Latched if machining does not complete in time; forces state 9.
Bases_Timeout	45 s TON in state 3	Same as lids but with a shorter timeout because the bases cycle is shorter in the simulator.
Pusher Return	5 s TON when pusher leaves back limit	Latched if pusher fails to return; flagged in assembly FB.
Robot Lost Item	Item-detect false during states 5–7 of robot sequence	Latched if grip is lost mid-traverse.

The recovery path is: operator presses HMI_FaultAck → Reset signal pulsed on the offending CNC → fault flags cleared → state returns to 0. For pusher and robot faults the operator must manually clear the obstruction in the scene before resetting.

12. Supervisory Control and HMI

A KTP700 Basic PN panel hosts the supervisory interface. The screens follow the layout used in the reference design:

- Root screen: Start, Stop, Reset buttons; current production counts; target inputs for lids and bases; switch between continuous and target mode; alarm display.
- Machining screen: per-machine ready/busy state, real-time conveyor speed, count display, and fault acknowledgement.
- Assembly screen: live count of finished green and blue products and a reset-counter button.

Every screen reads from DB_Machining and DB_Sorting so the HMI is a thin view layer over the same state that the FCs manipulate.

13. Testing and Validation

Testing was performed iteratively while building each station. The most informative tests came from watching the watch tables live during a run:

- Machining state progression was verified by watching Lids_State and Bases_State as parts moved through the cycle, with explicit checks that both AtEntry and AtExit transitioned correctly.
- Sorter alternation was verified by counting consecutive parts and confirming that the direction alternated within each colour.
- Exit queue priority was verified by deliberately scheduling both machines to finish at nearly the same time and confirming that lids waits while the bases part passes.
- Fault paths were exercised by manually setting MC1_Lids_HasError TRUE in the simulator and verifying that the line halted, the operator was alerted, and a Fault_Ack cleared the system back to idle.

During testing several timing issues surfaced — notably the timeout firing before the part reached the exit and the loading delay being too short. These were corrected by increasing PT values and by reordering the timer reset relative to the state change.

14. Challenges and Solutions

Challenge	Resolution
Vision sensor returned a single Bool that could not distinguish "green" from "empty."	Installed a second vision sensor and configured one for blue, one for green. The OR of the two is the part-present signal.
Bases CNC sometimes failed to assert Busy, leaving the state machine stuck at state 2.	Diagnosed as a Factory I/O signal-mapping inconsistency. Verified driver bindings and added a short hold-time before transitioning.
Parts collided at the merge between Lids and Bases exits.	Added two queue diffuse sensors and gave bases priority; lids waits if the bases queue sensor is high.
Output belt occasionally dropped a part on top of a queued one in front of the aligner.	Added an override on Belt_Output to gate it off when either alignment sensor sees a queued part.
Main exit conveyor never started.	Traced to a never-set AlwaysTRUE flag wired to FB_Assembly_Standalone.Run. Re-wired to System_Armed.
Both machines tried to claim the merge conveyor simultaneously, oscillating.	Removed Conveyor_Exit_Main writes from FC_Lids and FC_Bases. The assembly FB now owns that output exclusively.

15. Further Development

Several extensions are natural next steps:

- Replace the hard-coded timeout values with HMI inputs, allowing the operator to tune the line for slow versus fast simulation speeds without recompiling.
- Migrate the colour and routing logic into a structured DB array indexed by part ID, so adding a third or fourth colour does not require new FC code.

- Add an analog-speed control on the conveyors so the line slows to 50% when a single machine is busy rather than stopping outright when both are busy (closer to the original Olives reference design).
- Introduce a distributed-control overlay: a supervisory station that coordinates several PLCs each owning one cluster of the line, with a shared OPC UA namespace for the DB variables.
- Add encoder feedback on the conveyor motors so the system can pinpoint where a stalled part is, rather than relying on discrete diffuse sensors.

16. Conclusion

This phase delivered a complete integrated production line in which feeding, sorting, alignment, machining, and assembly run as a single coordinated machine. The work focused on the machining centre and its integration with the upstream and downstream stations, with three architectural decisions standing out as central.

First, exit-lane priority based on a small set of queue sensors is sufficient to prevent collisions at the merge — there is no need for a centralised scheduler. Second, lifting cross-cutting actuator assignments out of state-machine cases into top-of-FC global gating makes the state machine focus on what changes, not on what merely needs to be held. Third, shared run flags and a clear ownership policy for each actuator avoids the duelling-master bugs that plague otherwise-correct station code.

The line passes all the functional checks set out in the Phase 2 brief: parts emerge, are sorted by colour and routed evenly between the two machining centres, are machined and exited without pileups, and are assembled and counted with full fault visibility. The codebase is well-positioned for the further extensions discussed in Section 15.

17. Drive Link

https://drive.google.com/drive/folders/1rVqt_vwqgUfRUettfPZ0bc-P6G8OQ369?usp=sharing

Summative Assessment Task Model Answer / CAF

Course Code	MCT313			Course	Automation				Major Task	Date	7/5/2026					
Student Name:				Ahmed Mohamed Behiry						Student ID:		22P0044				
Mastery ≥90%				Accomplished ≥75 & <90%				Adequate ≥60 & <75%				Inadequate <60%				
100	96	93	90	89	84	79	75	74	69	64	60	59	40	20	0	
• Understand the theoretical background of the automation task • Design and build the simulated plant/production line • Design and implement a complete program to achieve the required process automation • Design and build an HMI interface to indicate the state of each station on the production line and show the field-related alarms • Comment on the overall project performance • Issue an excellent professional report indicating all results				• Understand the theoretical background of the automation task • Design and build the simulated plant/production line • Design and implement a complete program to achieve the required process automation • Design and partially build an HMI interface to indicate the state of each station on the production line and show the field-related alarms • Comment on the overall project performance • Issue a good professional report highlighting major results				• Understand the theoretical background of the automation task • Design and build the simulated plant/production line • Design and implement most of the program to achieve the required process automation • Do not manage to build an HMI interface to indicate the state of each station on the production line • Comment on the overall project performance • Issue a fair professional report highlighting some results				• Understand the theoretical background of the automation task • Design and build the simulated plant/production line • Design and partially implement the program to achieve the required process automation • Do not manage to build an HMI interface to indicate the state of each station on the production line • Comment on the overall project performance • Issue an incomplete report				
							✓									
First Marker Total				23		First Marker Signature				Dr. Mohamed Omar				ASU Agreed Mark		77
Second Marker Total				23		Second Marker Signature				Egy. Mina Maher				UEL Agreed Mark		61
General Comments: GOOD								UEL Grade Scale		Agreed Mark Range		ASU Grade Scale				
								% Equivalent at UEL				% at ASU		Grade		
								95% and higher		97% and higher		A+				
								82% to less than 95%		93% to less than 97%		A				
								70% to less than 82%		89% to less than 93%		A-				
								66% to less than 70%		84% to less than 89%		B+				
								63% to less than 66%		80% to less than 84%		B				
								60% to less than 63%		76% to less than 80%		B-				
								56% to less than 60%		73% to less than 76%		C+				
								53% to less than 56%		70% to less than 73%		C				
								50% to less than 53%		67% to less than 70%		C-				
								45% to less than 50%		64% to less than 67%		D+				
								40% to less than 45%		60% to less than 64%		D				
								Less than 40%		Less than 60%		F				